Django comes prepackaged with some other filters. You can dive deep into the documentation to see what is available or learn to further customize on your own: https://docs.djangoproject.com/en/dev/ref/contrib/admin/filters/.

When playing with your new list filter, take a look at the resulting URL. The first value in each `.lookups() tuple` gets used as the query parameter. The query parameter itself is defined through the `parameter_name` attribute. All of this is still a web application—Django is just doing many things on your behalf. Because the Django Admin uses query parameters to modify what is displayed on the page, you can take advantage of this and add features for yourself, you'll do just that in the next section.

5.2.3 New in Django 5: Facets

If you're using Django 5, you may have noticed another link in the filter control box on the right-hand side of the page: Show Counts. When you click this link, each filter choice has a counter appended, showing how many items are included in that filter. These counts are called *facets*. Clicking the link adds a `__facets=True` query parameter to the URL. When facets are being displayed, Show Counts becomes Hide Counts.

You can control whether facets are displayed by adding a `show_facets` member to your admin model. There are three possible settings:

- `show_facets = admin.ShowFacets.ALWAYS`—Always display the facet.
- `show_facets = admin.ShowFacets.ALLOW`—Use the Show and Hide links, displaying facets if the query parameter is present (this is the default).
- `show_facets = admin.ShowFacets.NEVER`—Disable facets.

Displaying the facet information means extra queries to the database because the Django Admin has to look up the count of each filter. This is why the `NEVER` option is available, so you can turn it off if the query is too expensive for your page.

5.3 Cross-linking between related objects

So far, you've only added `Musician` to the Django Admin. Adding a `Band` object is no different but has the added wrinkle of the `BandModel` including a relationship with the `Musician` class. To see how the Django Admin deals with inter-object relationships, create a bare minimum `BandAdmin` class, like the one in the following listing.

Listing 5.5 `BandAdmin` class

```
...
from bands.models import Musician, Band
...
@admin.register(Band)
class BandAdmin(admin.ModelAdmin):
    pass
```

Add the Band class to the import.

Define and register a minimum BandAdmin class.

With your new class in place, run the development server, visit the Band Listing page, and then click on Wishful Thinking. Figure 5.10 shows the Change page for the object, including a multiselect widget for choosing the related `Musician` objects.